

# INFORMATICS INSTITUTE OF TECHNOLOGY

IN COLLABORATION WITH

UNIVERSITY OF WESTMINSTER.

## **5SENG003C.2 Algorithms: Theory, Design and Implementation.**

**Student name** - Mohommed Abdul Malik

**UOW ID** – w2121305

**IIT ID** – 20241120

## a). Data Structure and Algorithm selection.

The directed graph was represented using an **adjacency list**, along with the following data structures:

- **Adjacency List** - stores all outgoing edges for each vertex.
- **Reverse adjacency list** - stores all incoming edges for each vertex.
- **Outdegree map** – stores number of outgoing edges for each vertex.

This approach allows quick traversal through the graph and quick updates when vertices are removed. From using the outdegree map, we can identify sink vertices within constant time. It is essential for sink elimination process.

Rather than choosing an adjacency matrix, we use this approach because it is more space efficient. It requires  $O(v + e)$  space instead of  $O(v^2)$ . Thus, making it suitable for sparse graphs.

### **CheckAcyclic.java Class.**

In this class, the sink elimination algorithm was used to determine if the directed graph is acyclic or not. The sink elimination algorithm iteratively eliminates the vertices that have no outgoing edges (sinks). After removing all sinks, if no vertices are remained, then the graph is acyclic. Otherwise, graph is cyclic (a cycle exists). This runs in  $O(v + e)$  time. Therefore, this approach is efficient.

### **FindCyclic.java Class.**

In this class, a depth first search (DFS) algorithm is used to determine and output an actual cycle within the cyclic graph. This approach tracks visited nodes and a recursion stack to detect back edges. Those back edges are the indicators of cycles. Once a back edge is found, the cycle is reconstructed using the parent pointers.

In conclusion, using a combination of an adjacency list, sink elimination and DFS is an effective and scalable method for detecting and extracting cycles from directed graphs.

## b). Benchmark examples.

The following benchmark examples were used to evaluate the implemented solution. An acyclic and cyclic graph was tested using `acyclic/a_40_0.txt`, `cyclic/c_40_0.txt` files respectively. These examples represent how the algorithm identifies acyclic graphs and cycle detection by producing the appropriate outputs. The outputs are as follows.

### Example 1: Acyclic Graph.

```
File - Main
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\
Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3
.2\lib\idea_rt.jar=53734:C:\Program Files\JetBrains\IntelliJ
IDEA Community Edition 2024.3.2\bin" -Dfile.encoding=UTF-8 -
Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -
classpath F:\Algo\CW\Algo_CW\out\production\Algo_CW Main
===== Welcome to cycle finder! =====
Please enter the file name containing your graph data (ex:
acyclic/a_40_0.txt): acyclic/a_40_0.txt

=====
sink elimination process
=====

Sink found : 24 ...Sink Eliminated
Sink found : 19 ...Sink Eliminated
Sink found : 23 ...Sink Eliminated
Sink found : 35 ...Sink Eliminated
Sink found : 1 ...Sink Eliminated
Sink found : 22 ...Sink Eliminated
Sink found : 30 ...Sink Eliminated
Sink found : 39 ...Sink Eliminated
Sink found : 8 ...Sink Eliminated
Sink found : 2 ...Sink Eliminated
Sink found : 25 ...Sink Eliminated
Sink found : 36 ...Sink Eliminated
Sink found : 34 ...Sink Eliminated
Sink found : 17 ...Sink Eliminated

Page 1 of 3
```

```
File - Main
Sink found : 29 ...Sink Eliminated
Sink found : 15 ...Sink Eliminated
Sink found : 6 ...Sink Eliminated
Sink found : 4 ...Sink Eliminated
Sink found : 26 ...Sink Eliminated
Sink found : 27 ...Sink Eliminated
Sink found : 9 ...Sink Eliminated
Sink found : 32 ...Sink Eliminated
Sink found : 33 ...Sink Eliminated
Sink found : 18 ...Sink Eliminated
Sink found : 21 ...Sink Eliminated
Sink found : 0 ...Sink Eliminated
Sink found : 28 ...Sink Eliminated
Sink found : 10 ...Sink Eliminated
Sink found : 31 ...Sink Eliminated
Sink found : 20 ...Sink Eliminated
Sink found : 37 ...Sink Eliminated
Sink found : 7 ...Sink Eliminated
Sink found : 5 ...Sink Eliminated
Sink found : 16 ...Sink Eliminated

Page 2 of 3
```

```
File - Main
Sink found : 12 ...Sink Eliminated
Sink found : 11 ...Sink Eliminated
Sink found : 38 ...Sink Eliminated
Sink found : 14 ...Sink Eliminated
Sink found : 13 ...Sink Eliminated
Sink found : 3 ...Sink Eliminated

Graph is acyclic: YES
Elimination order: [24, 19, 23, 35, 1, 22, 30, 39, 8, 2, 25, 36
, 34, 17, 29, 15, 6, 4, 26, 27, 9, 32, 33, 18, 21, 0, 28, 10,
31, 20, 37, 7, 5, 16, 12, 11, 38, 14, 13, 3]

Do you want to check another file? (y/n):

Page 3 of 3
```

Therefore, the algorithm successfully finds and remove sink vertices until no vertices remain. This confirms that there are no cycles in this graph.

### Example 2: Cyclic Graph.

```
File - Main
"C:\Program Files\Java\jdk-23\bin\java.exe" "-javaagent:C:\
Program Files\JetBrains\IntelliJ IDEA Community Edition 2024.3
.2\lib\idea_rt.jar=57319:C:\Program Files\JetBrains\IntelliJ
IDEA Community Edition 2024.3.2\bin" -Dfile.encoding=UTF-8 -
Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -
classpath F:\Algo\CW\Algo_CW\out\production\Algo_CW Main
===== Welcome to cycle finder! =====
Please enter the file name containing your graph data (ex:
acyclic/a_40_0.txt): cyclic/c_40_0.txt

=====
sink elimination process
=====

Sink found : 27 ...Sink Eliminated
Sink found : 38 ...Sink Eliminated
Sink found : 14 ...Sink Eliminated
Sink found : 11 ...Sink Eliminated
Sink found : 19 ...Sink Eliminated
Sink found : 15 ...Sink Eliminated
Sink found : 36 ...Sink Eliminated
Sink found : 30 ...Sink Eliminated
Sink found : 1 ...Sink Eliminated
Sink found : 5 ...Sink Eliminated

No more sinks available.
Graph is acyclic: NO

remaining vertices (cycle part) : [0, 2, 3, 4, 6, 7, 8, 9, 10,
12, 13, 16, 17, 18, 20, 21, 22, 23, 24, 25, 26, 28, 29, 31, 32
, 33, 34, 35, 37, 39]

Page 1 of 2
```

```
File - Main
A cycle was found in the graph. The cycle path is:
10 -> 9 -> 37 -> 7 -> 39 -> 25 -> 3 -> 24 -> 10

Do you want to check another file? (y/n):

Page 2 of 2
```

After removing sinks (27, 38, 14, 11, 19, 15, 36, 30, 1 and 5), no more sinks exist. Therefore, graph is not acyclic. Then the DFS algorithm is used to determine and reconstruct the cycle. This results in the cycle:  $10 > 9 > 37 > 7 > 39 > 25 > 3 > 24 > 10$

## c). Performance analysis.

The performance of the algorithm evaluated using empirical analysis and theoretical reasoning. From this we determine how the runtime of the algorithm scales with doubling input size, and we clarify algorithm's order of growth using big-O notation.

### Empirical analysis (Doubling hypothesis).

The runtime of the algorithm was tested using graphs of increasing size. The recorded runtimes as follows:

| Number of vertices | Runtime for acyclic (ms) | Runtime for cyclic |
|--------------------|--------------------------|--------------------|
| 40                 | 10                       | 10                 |
| 80                 | 22                       | 16                 |
| 160                | 30                       | 10                 |
| 320                | 24                       | 22                 |
| 640                | 24                       | 27                 |
| 1280               | 24                       | 28                 |
| 2560               | 50                       | 36                 |
| 5120               | 52                       | 48                 |
| 10240              | 59                       | 64                 |

According to this, runtimes represent a general increase as the number of vertices doubled. But the increase is not perfectly consistent because of system level variations. Although the overall pattern represents a gradual and approximately linear growth in the runtime.

### Theoretical analysis

The algorithm consists of two main components.

#### I. Sink elimination algorithm

Each vertex is eliminated at most once (if graph is acyclic). And each edge is processed when updating the outdegree. Therefore, the time complexity for this process is  $O(V + E)$ .

#### II. Cycle detection

If the graph is cyclic, DFS is used to detect and reconstruct a cycle. DFS visits each and every vertex and edge maximum of once. Therefore, the time complexity for this process is also  $O(V + E)$ .

Since both of these components are linear in terms of edges and vertices, the overall time complexity of this solution is  $O(V + E)$ .

In conclusion, performance analysis confirms that the implemented solution represents linear growth. Therefore, the solution can be classify as having an order of growth of  $O(V + E)$ , ensuring solution is efficient and scalable for larger inputs.